LAPORAN PRAKTIKUM STRUKTUR DATA

“STACK”

Disusun Oleh :

Dzhillan Dzhalila

2511531001

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid A.

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur penulis panjatkan atas kehadirat Allah SWT Tuhan Yang Maha Esa yang telah memberikan rahmat dan berkatnya, sehingga penulis dapat menyelesaikan laporan praktikum ini yang telah dilaksanakan sebagai pertemuan ke-3 untuk mata kuliah Praktikum Struktur Data. Praktikum ini berfokus pada pembahasan konsep struktur data Stack dengan implementasi linkedList dan ArrayList.

Melalui praktikum ini, hasil yang diharapkan adalah penulis dapat memahami dan mengimplementasikan penggunaan Stack dalam real Project. Melalui projek projek sederhana yang dilakukan pada praktikum ini membantu mengembangkan pemahaman akan Stack pada program.

Penulis menyadari bahwa, laporan praktikum ini masih memiliki banyak kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan dan pengembangan di masa mendatang. Semoga laporan ini dapat memberikan manfaat, baik sebagai bahan pembelajaran penulis maupun referensi pembaca dalam memahami dasar-dasar pemrograman.

Padang, 2026

Dzhillan Dzhalila

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 3
1.1 Latar Belakang 3
1.2 Tujuan 3
1.3 Manfaat 3
BAB II PEMBAHASAN 4
2.1 Dasar Teori 4
2.2 Program stackArray_2511531001 5
2.3 Program stackArrayDriver_2511531001 6
2.4 Program StackPostFix_2511531001 7
2.5 Program NilaiMaksimum_2511531001 9
2.6 Program SiswaStack_2511531001 11
BAB III KESIMPULAN 13
DAFTAR PUSTAKA 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman bahasa Java, salah satu struktur data yang dapat digunakan adalah Stack. Stack sendiri sering digunakan untuk menyimpan data dimana memungkinkan data terakhir kali masuk akan menjadi data pertama kali keluar. Ini menjadikan urutan data yang dimasukkan ke dalam tumpukan stack akan menjadi terbalik ketika tumpukan tersebut dikeluarkan.

Dalam Komputer Science konsep stack sendiri digunakan dalam berbagai kebutuhan. Salah satunya adalah digunakan untuk memabangun algoritma yang canggih. Seperti algoritma untuk mencari jalan keluar dari labirin dengan menyusuri labiran tetapi dimulai dari jalan keluarnya (backtracking). Tidak hanya itu, stack juga sering digunakan dalam memproses suatu data. Stack dapat membantu mengetahui apakah sebuah string adalah palindrom atau bukan, dan juga dapat mengubah ekspresi infix menjadi postfix atau prefix.

1.2 Tujuan

1. Memahami konsep dasar dari struktur data Stack.
2. Mengidentifikasi penggunaan Stack dalam program.
3. Mengimplementasikan metode bawaan dari Stack.

1.3 Manfaat

1. Memberikan pengalaman langsung dalam mengimplementasikan Stack.
2. Melatih ketepatan dalam memilih struktur data untuk menghasilkan program yang lebih efisien dan terstruktur

BAB II
PEMBAHASAN

2.1 Dasar Teori

Stack adalah sebuah koleksi yang berbasis metode penambahan elemen dan pengembalian elemen elemen di dalamnya dengan urutan yang terbalik atau biasa disebut LIFO (Last in First Out). Elemen elemen yang dimasukkan ke dalam Stack memiliki urutan penyisipan, tetapi kita tidak bisa menyebut elemen tersebut memiliki indeks. Operasi penambahan dan penghapusan elemen hanya dapat dilakukan pada elemen yang berada paling atas dari tumpukan atas stack (elemen yang terakhir kali ditambahkan).

Stack terbagi menjadi 2 yaitu stack yang statis (tidak berubah) dan juga stack dinamis (stack yang dapat berubah). Stack dinamis biasanya diterapkan dengan LinkedList sehingga memungkinkan akses kapasitas yang tak tetap. Ada beberapa class bawaan yang sudah tersedia pada sturktur data, yaitu :

a. Stack <E> () : membuat satu stack baru dengan tipe data elemennya "E"
b. push (value) : menempatkan nilai yang diberikan di posisi teratas stack
c. pop () : meremove nilai teratas dari satck dan mereturnnya
d. note : akan muncul output "EmptyStackException" jika stacknya kosong
e. peek () :returns nilai teratas dari stack tanpa menghapusnya
f. size () : menghasilkan jumlah dari element di dalam stack
g. isEmpty () : mengembalikan nilai true jika stack tersebut kosong

Dan adapun inisiasi umum untuk struktur data ini adalah sebagai berikut :

```
Stack<Integer> s = new Stack<Integer>();
```

2.2 Program stackArray_2511531001

2.2.1 Kode Program

```
package pekan3_2511531001;
import java.util.*;
public class stackArray_2511531001 {
  static final int MAX_1001 = 1000;
  int top_1001;
  int[] a_1001 = new int [MAX_1001]; //array
  boolean isEmpty_1001() {
        return (top_1001 <0);
  }
  stackArray_2511531001(){
        top_1001=-1;
  }
  boolean push_1001(int x_1001) {
        if (top_1001 >=(MAX_1001 -1)) {
              System.out.println("Stack Overflow");
              return false;
        }
        else {
              a_1001[++top_1001] = x_1001;
              System.out.println( x_1001 +" dimasukkan
dalam stack");
              return true;
        }
  }
  int pop_1001 () {
        if (top_1001 <0) {
              System.out.println("Stack Underflow");
              return 0;
        } else {
              int x_1001 = a_1001 [top_1001--];
              return x_1001;
        }
  }
  int peek_1001 () {
        if (top_1001<0) {
              System.out.println("Stack Underflow");
              return 0;
        }
        else {
              int x_1001= a_1001 [top_1001];
              return x_1001;
        }
  }
  void print_1001 () {
        for (int i_1001 = top_1001; i_1001 >-1; i_1001 --) {
              System.out.print(" "+ a_1001 [i_1001]);}}}
```

Kode Program 2.1 : program Stack yang menggunakan array

2.2.2 Penjelasan

Pertama program ini mengimpor library java karena kita menggunakan Stack. Setelah itu mendefinisikan kapasitas

maksimum Stack itu 1000 elemen dengan kelas bertipe final untuk memastikan nilai tidak berubah Pendeklarasian int top_1001 berfungsi sebagai variabel penunjuk yang menyimpan indeks elemen paling atas. Jika variabel top_1001 bernilai -1 maka stack dalam kondisi kosong.

Lalu program menggunakan konstruktor stackArray_2511531001 dan membuat method validator yaitu isEmpty_1001() yang akan mengembalikan nilai true jika top_1001 <0. Selanjutnya ada method yang menggunakan operasi utama (Push, Pop, Peek). Berikut rincian dari penggunaan masing masing operasi

1. push_1001(int x_1001) : bertipe Boolean yang akan mendeteksi stack overflow terlebih dahulu. Jika stack belum penuh maka method ini akan menambahkan elemen ke dalam stack.
2. pop_1001(i) : bertipe int yang akan mengambil dan menghapus elemen teratas jika stack tidak kosong atau tidak *underflow*. Method ini juga akan menampilkan pesan "Stack Underflow" jika stack kosong dan tidak ada elemen yang bisa dihapus dan diambil.
3. peek_1001() : bertipe int yang berfungsi untuk melihat elemen yang berada paling atas stack.

Setelah kelas void print_1001() yang akan menampilkan seluruh isi stack. Perulangan for dimulai dari top_1001 dan berjalan mundur hingga mencapai indek 0. Urutan cetak akan mencermikan urutan tumpukan dari atas ke bawah.

2.3 Program stackArrayDriver_2511531001

2.3.1 Kode Program

```
public class stackArrayDriver_2511531001 {
      public static void main (String [] args) {
            stackArray_2511531001 s_1001 = new
stackArray_2511531001 ();
            s_1001.push_1001(10);
            s_1001.push_1001(20);
            s_1001.push_1001(30);
```

```
            System.out.println(s_1001.pop_1001() + "
dikerluarkan dari stack");
            System.out.println("Elemen teratas adalah :"
+s_1001.peek_1001());
            System.out.print("Elemen pada stack:");
            s_1001.print_1001();
        }}
```

Kode Program 2.2 : kode driver StackArray

2.3.2 Penjelasan Program

Mendeklarasikan nama variabel untuk stack yaitu s_1001 untuk langkah pertama. Setelah itu menggunakan kelas method *push_1001* untuk menambahkan 3 elemen yaitu 10,20, dan 30. Lalu menggunakan method *pop_1001* untuk menghapus elemen teratas pada stack. Selanjutnya, menggunakan perintah *print_1001* untuk menampilkan elemen stack yang tersisa.

2.3.3 Output

```
<terminated> stackArrayDriver_2511531001 [Ja
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikerluarkan dari stack
Elemen teratas adalah :20
Elemen pada stack: 20 10
```

Gambar 2.1 : Output program driver stackArray

Dapat dilihat dari hasil elemen pada stack hanya tertampil 20 dan 10 karena elemen “30” sudah dihapus dengan perintah *pop_1001* sebelumnya.

2.4 Program StackPostFix_2511531001

2.4.1 Kode Program

```
package pekan3_2511531001;
import java.util.*;
public class StackPostFix_2511531001 {
public static int postfixEvaluate_1001 (String
expression_1001) {
 Stack<Integer> s_1001 = new Stack <Integer> ();
 Scanner input_1001 = new Scanner (expression_1001);
 while (input_1001.hasNext()) {
        if (input_1001.hasNextInt()) {
```

```
                s_1001.push(input_1001.nextInt());
        }else {
                String operator_1001 = input_1001.next();
                int operand2_1001 = s_1001.pop ();
                int operand1_1001 = s_1001.pop();
                if(operator_1001.equals("+")) {
                        s_1001.push (operand1_1001 +
operand2_1001);
                } else if (operator_1001.equals("-")) {
                        s_1001.push(operand1_1001 -
operand2_1001);
                }  else if (operator_1001.equals("*")) {
                        s_1001.push(operand1_1001 *
operand2_1001);
                } else {
                        s_1001.push(operand1_1001 /
operand2_1001);
                }
        }
 }
 input_1001.close();
 return s_1001.pop();
}

public static void main (String [] args) {
 System.out.println("hasil postfix= "+
postfixEvaluate_1001("5 2 4 * + 7 -"));
}
}
```

2.4.2 Penjelasan Program

Program ini diawali dengan struktur kelas dan method utama. Method static postfixEvaluate_1001(String expression_1001) akan menerima string ekspresi postfix sebagai inout dan mengembalikan hasil perthitungan bertipe int. Setelah itu inisiasi Stack yang bernama s_1001 untuk menyimpan operand(angka) sementara selama masa proses pengecekan. Dikarenakan kita ingin meminta inpun kepada user, kita juga menggunakan Scanner.

Setelah itu kita menggunakan perulangan untuk melakukan pengecekan kepada inputan user. Perulangan kali ini menggunakan perulangan while yang akan berjalan selama masih ada inputan user yang belum diproses dalam ekspresi. Lalu diberikan percabangan, jika data inputan user berupa angka int atau operand maka data tsb akan dimasukkan ke dalam stack s_1001 menggunakan perintah

push_1001. Namun jika kondisi tsb tidak terpenuhi maka program akan memeriksa kemungkinan kondisi kedua.

Kondisi kedua adalah ketika inputan user yang diperiksa pada perulangan berupa operator aritmatika kabataku(*, /, +, -), maka 2 operand teratas akan dioperasikan dengan operator yang ditemukan. Dua data teratas terlebih dahulu sudah dikeluarkan dengan perintah *pop_1001*, dimana elemen pertama yang dikeluarkan akan menjadi operand2 dan elemen kedua yang dikeluarkan akan menjadi operand1. Setelah itu, percabangan if-else if akan memeriksa operator mana yang ditemui dan akan mengaplikasi operator yang sesuai kepada 2 operand tadi. Setelah itu, hasil operasi antara keduanya akan ditambahkan kembali ke dalam ke stack dengan perintah *push_1001*.

2.4.3 Output

```
<terminated> StackPostFix_251153
hasil postfix= 6
```

Hasil ini dihasilkan dari program main pada program di atas. Pada program main, method postfixEvaluate dipanggil dan dari ekspresi postfix 5 2 4 * + 7 -"

2.5 Program NilaiMaksimum_2511531001

2.5.1 Kode Program

```
package pekan3_2511531001;
import java.util.Stack;
public class NilaiMaksimum_2511531001 {
 public static int max (Stack <Integer> s_1001) {
        Stack<Integer> backup_1001 = new Stack <Integer>();
        int maxValue_1001 = s_1001.pop ();
        backup_1001.push((maxValue_1001));
        while (!s_1001.isEmpty()) {
               int next_1001 = s_1001.pop();
               backup_1001.push (next_1001);
               maxValue_1001 = Math.max(maxValue_1001,
next_1001);
        }
        while(backup_1001.isEmpty()) {
               s_1001.push(backup_1001.pop());
```

```
        } return maxValue_1001;
 }

 public static void main (String [] args) {
        Stack <Integer> s_1001 = new Stack <Integer> ();
        s_1001.push(70);
        s_1001.push(12);
        s_1001.push(20);
        System.out.println("isi stack " + s_1001);
        System.out.println("Stack Teratas " +
s_1001.peek());
        System.out.println("Nilai Maksimum " + max(s_1001));
 }

}
```

2.5.2 Penjelasan

Program ini bertujuan untuk mencari nilai terbesar dari nilai yang dimasukkan ke dalam Stack. Algoritma dari program ini adalah pertama membuat stack cadangan untuk menyimpan elemen elemen yang dikeluarkan dari stack utama dengan tujuan agar stack asli dapat dikembalikan ke keadaan semula setelah proses selesai.

Logika program ini adalah untuk mengambil nilai yang pertama dimasukkan ke dalam stack dan menjadikannya sebagai nilai pembanding awal (int maxValue_1001 = s_1001.pop()). Elemen pembanding awal tsb akan langsung disimpan ke stack backup dan akhirnya melakukan perulangan dengan while selama stack backup_1001 blm kosong.

2.5.3 Output

```
<terminated> NilaiMaksimum_2511531001 [
isi stack [70, 12, 20]
Stack Teratas 20
Nilai Maksimum 70
```

Output ini adalah dihasilkan dari program main yang menambahkan ketiga elemen (70,12,20) dan isi stack ditampilkan. Menggunakan method peek untuk menampilkan elemen teratas dan

memanggil method NilaiMaksimum_2511531001 untuk menampilkan nilai terbesar dari 3 elemen

2.6 Program SiswaStack_2511531001

2.6.1 Kode Program

```
package pekan3_2511531001;
import java.util.*;
class Siswa_1001 {
  String nama_1001;
  int nim_1001;
  // mendeklarasikan type dari program

  public Siswa_1001 (String nama_1001, int nim_1001) {
        this.nama_1001 = nama_1001;
        this.nim_1001 = nim_1001;
  } // konstruktor untuk komponen nama dan nim

  @Override
  public String toString () {
        return "NIM : "+ nim_1001 + ", Nama : " + nama_1001;
  } // mengembalikan kompoonen ke tipe data string
}
public class SiswaStack_2511531001 {
  private ArrayList <Siswa_1001> stack_1001;
  // menginisiasi ArrayList dengan nama stack_1001 dengan
parameter siswa_1001

  public SiswaStack_2511531001() {
        stack_1001 = new ArrayList<>();
  }
  // mendeklarasikan stack_1001 sebagai ArrayList yang baru

  public void push_1001 (Siswa_1001 mhs_1001) {
        stack_1001.add(mhs_1001);}
  // menambahkan inputan data dari mhs_1001 ke stack_1001

  public Siswa_1001 pop_1001() {
        if (!isEmpty_1001()) {
              return stack_1001.remove (stack_1001.size() -
1);
        }
        return null;}
  // menghapus elemen pada stack_1001 jika stack tidak kosong

  public Siswa_1001 peek_1001 () {
        if (!isEmpty_1001()) {
              return stack_1001.get(stack_1001.size() -1);
        }
        return null;}
  // menampilkan elemen teratas pada stack_1001

  public boolean isEmpty_1001() {
        return stack_1001.isEmpty();}
```

```
  // mengembalikan nilai true jika stack_1001 kosong

  public void tampilkanSiswa_1001() {
        for ( int i_1001 = stack_1001.size () -1 ;
i_1001 >=0; i_1001 --) {
              System.out.println(stack_1001.get(i_1001));
        }}
  //perulangan untuk menampilkan semua elemen stack_1001

  public static void main (String [] args) {
        SiswaStack_2511531001 studentStack_1001 = new
SiswaStack_2511531001 ();
        // memanggil kelas stack dan mendeklarasikan
studentStack_1001 menjadi new Stack

        Siswa_1001 mhs1_1001 = new Siswa_1001 ("Ali" , 1);
        Siswa_1001 mhs2_1001 = new Siswa_1001 ("Bobby" , 2);
        Siswa_1001 mhs3_1001 = new Siswa_1001 ("Charles" ,
3);


        studentStack_1001.push_1001((mhs1_1001));
        studentStack_1001.push_1001((mhs2_1001));
        studentStack_1001.push_1001((mhs3_1001));
        // menambahkan elemen nama siswa Ali, Bobby, Charles

        System.out.println("Siswa di dalam stack :");
        studentStack_1001.tampilkanSiswa_1001();
        //menampilkan nama siswa

        System.out.println("Siswa teratas " +
studentStack_1001.peek_1001());
        // menampilkan siswa teratas pada stack

        System.out.println("mengeluarkan siswa teratas " +
studentStack_1001.pop_1001());
        // menghapus nama siswa teratas studentStack dengan
method pop_1001

        System.out.println("Daftar siswa setelah di pop: ");
        studentStack_1001.tampilkanSiswa_1001();
        // menampilkan semua siswa yang masih ada di stack
  }
```

BAB III
KESIMPULAN

Dari praktikum yang telah dilaksanakan ada beberapa hal yang dapat disimpulkan

1. Mengimplementasikan struktur data Stack dengan Array mengharuskan kita untuk mendeklarasikan variabel logika seperti push/pop/ peek ataupun pengecekan seperti isEmpty, overflow dan underflow secara manual. Hal ini disebabkan oleh Array yang tidak memiliki fitur bawaan dan hanya menyediakan blok memori berurutan saja.
2. Penerapan dari prinsip Last In First Out (LIFO) bisa digunakan untuk menyelesaikan masalah matematika postfix dan menentukan nilai maksimal.
3. Visualisasi data pada method tampilkanSiswa_1001() menunjukkan bahwa urutan tumpukan sangat diperhatikan pada stack. Semua elemen yang muncul pertama kali akan selalu dari elemen yang berada pas stack.

DAFTAR PUSTAKA

[1] Oracle Corporation, "Stack (Java Platform SE 8)," *Java SE 8 Documentation*, Oracle. [Online]. Tersedia: https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html. [Diakses: 14-Apr-2026].

[2] "Stack Implementation Using Array," *Simplilearn*. [Online]. Tersedia: https://www.simplilearn.com/tutorials/data-structure-tutorial/stack-implementation-using-array. [Diakses: 14-Apr-2026].